

WILLIS PORT

Web Application - Technical, Security & MVP Project Brief

Working project reference - August 2026

Purpose. This document summarizes the agreed direction for the first Willis Port web application: a secure system for capturing customer service requests, calculating shipping charges, generating invoices, recording payments, and issuing receipts.

1. Project Scope

- Capture requests received through Email, WhatsApp, Phone Calls, Walk-ins, and the Willis Port website.
- Create and maintain customer records and shipment/service request records.
- Capture type of goods, service category, weight (kg), CBM, supporting image/PDF documents, and invoice-valid-until date.
- Support Normal Goods, Special Goods, Air Special, and other agreed shipping/service categories.
- Apply configured pricing rules and approved exchange rates; calculate in USD where required and charge the final amount in Ghana cedis (GHS).
- Generate professional PDF invoices and send/share them with customers.
- Track invoice status, including draft, sent, partially paid, paid, overdue, and cancelled.
- Record payments and generate receipts after payment.
- Provide management reporting: invoices sent, paid/unpaid amounts, revenue, and outstanding balances.

2. Recommended Technical Stack

Layer	Technology	Purpose
Frontend	Next.js + React	Web application and staff dashboard
Language	TypeScript	Safer JavaScript with type checking
Styling	CSS Modules	Willis Port white, black and orange UI
Backend/API	Node.js + Express + TypeScript	Business rules, calculations and secure APIs
Database	PostgreSQL	Customers, shipments, invoices, payments and reports
ORM	Prisma	Safer database access and schema management
Authentication	Firebase Authentication	Staff sign-in and identity management
File Storage	Google Cloud Storage / Firebase Storage	Invoices, receipts, images and PDFs
Hosting	Firebase App Hosting + Google Cloud Run	Frontend and backend hosting
PDF Generation	PDFKit or Puppeteer	Invoice and receipt PDF generation
Integrations	Email + WhatsApp Business API + exchange-rate API	Customer communication and currency conversion
Monitoring	Google Cloud Logging/Monitoring	Production logs, errors and operational visibility
Source Control	Git + GitHub	Version control and deployment workflow

Core data relationship: Customer -> Request -> Shipment/Cargo -> Invoice -> Payment -> Receipt.

3. Core Business & Pricing Principles

- Pricing rules are stored in the backend/database; they are not hard-coded into the browser.

- Example agreed rule: Normal Goods may use 260 kg = 1 CBM, subject to Willis Port confirmation and future configuration.
- The backend is the authoritative calculator for CBM, service price, exchange rate, invoice total, paid amount, and balance.
- The exact pricing rule and exchange rate used must be saved on each invoice so historical invoices do not change when rates change later.
- Finalized invoices should not be silently overwritten. Corrections or overrides require authorization and an audit record.

4. Security Considerations & Mitigation

Risk	Willis Port Example	Mitigation
Unauthorized access	Staff changing prices or invoices	Backend role-based authorization; least privilege; admin-only sensitive actions
Account takeover	Stolen staff credentials	Firestore Authentication; MFA for privileged users where possible; secure sessions
Broken access control / IDOR	User guesses another invoice ID	Authorize every sensitive request and object server-side
SQL injection	Malicious customer input	Prisma parameterized queries; no unsafe SQL concatenation
Invalid or malicious input	Text submitted as weight or extreme values	Strict server-side schema validation, formats, ranges and length limits
Invoice/payment tampering	Browser submits fake total or paid amount	Server calculates financial values; do not trust browser totals
Rate manipulation	User changes exchange rate in browser	Approved rate selected and stored server-side
XSS / browser attacks	Script inserted into notes or names	Output encoding, input constraints, Content Security Policy and security headers
API abuse / brute force	Automated repeated requests	Rate limiting, monitoring and abuse controls
Secret leakage	Database/API key exposed in frontend or Git	Backend-only secrets; Google Secret Manager/environment configuration; never commit secrets
Document exposure	Public invoice or shipment files	Private cloud storage; authenticated/authorized access or short-lived signed URLs
Malicious uploads	Unsafe PDF/image files	Allowlisted file types, MIME/extension/size checks, isolated storage
Data loss	Database deletion or corruption	Automated backups and tested restore procedure
Insider misuse	Unauthorized price override	Immutable audit logs recording who, what, when, before/after and reason
Error leakage	Stack trace sent to customer	Centralized safe error handling and internal logging
Transport interception	Customer/payment data sent insecurely	HTTPS/TLS everywhere
Dependency vulnerability	Compromised npm package	Lockfiles, dependency scanning, controlled upgrades and minimal dependencies

Security rule: Security is part of each feature definition, not a final phase. For every feature we identify the data, threat, allowed role, validation, authorization, audit requirements and failure cases before completion.

5. Initial User Roles

- ADMIN - manage users, pricing, exchange rates, permissions, reports and all operational records.
- OPERATIONS - create customers/requests, capture cargo details, manage shipment information and prepare invoices.
- ACCOUNTS - review invoices, record payments, generate receipts and access financial reports.

6. Delivery Scope in Phases

Phase	Primary Deliverables	Outcome
Phase 1 - MVP / Client Demo	Basic staff web app; customer/request capture; cargo details; configurable pricing; invoice calculation; PDF invoice; invoice	A working end-to-end demonstration Willis Port can review within a few days

	list/status; basic security and validation	
Phase 2 - Payments & Receipts	Payment capture; partial/full payment; balance tracking; receipt generation; payment history; accounts workflow	Complete invoice-to-payment operational flow
Phase 3 - Communications & Channels	Email sending; WhatsApp integration; web request form; channel/source tracking; customer notifications	Requests and documents flow through real customer channels
Phase 4 - Management & Controls	Role management; pricing/exchange-rate administration; advanced audit logs; dashboard/reporting; exports	Operational control and management visibility
Phase 5 - Production Hardening	Backups/restore testing; monitoring; rate limiting; security headers; upload hardening; performance; deployment pipeline; user acceptance testing	Production-ready release

7. First MVP - Deliverable for Client Review

Goal: Demonstrate the core Willis Port workflow quickly, using real-looking sample data, without waiting for every external integration.

1. Staff opens the Willis Port application and reaches a simple protected workspace.
2. Staff creates/selects a customer and records the request source: Email, WhatsApp, Phone, Walk-in, or Internet/Home.
3. Staff enters service/goods type, kilograms and/or CBM, notes, validity date, and optional document/image reference.
4. System applies a configured pricing rule and exchange rate and calculates the invoice in GHS.
5. Staff reviews and generates a professional Willis Port PDF invoice.
6. Invoice appears in an invoice list with customer, date, amount and payment/status information.
7. Client can review the workflow and approve business rules before payment, receipt and communication integrations are completed.

MVP Included

- Willis Port branded web interface (white, black and orange).
- Basic staff authentication/protected access.
- Customer and request creation.
- Request source/channel capture.
- Cargo/service fields: goods type, service type, kg, CBM and notes.
- Configurable sample pricing and USD/GHS exchange rate.
- Backend-authoritative invoice calculation.
- Invoice number, issue date and valid-until date.
- PDF invoice generation.
- Invoice list with draft/sent/paid or unpaid status basics.
- Server-side input validation, authorization baseline and audit-friendly data structure.

Deferred from the First Demo

- Live WhatsApp Business integration.
- Automated email delivery.
- Live market exchange-rate integration (a controlled/manual rate can be used for the MVP).
- Full payment gateway integration.
- Advanced financial reporting and exports.
- Complex multi-level approval workflows.
- Full customer self-service portal.

8. MVP Definition of Done

- A staff user can complete the core request-to-invoice flow without developer intervention.

- The invoice calculation is performed and validated on the backend.
- An invoice PDF can be generated with customer, cargo, pricing, exchange-rate and validity information.
- Data persists in the database and can be viewed again after refresh/restart.
- Unauthorized users cannot perform protected operations.
- Invalid input is rejected safely and useful errors are shown without leaking internal system details.
- The client can test the workflow and provide feedback on pricing rules, terminology, invoice layout and operational steps.

9. Development Approach

Learning/building principle: Write the feature first, understand the concept inside that feature, then move on.

Engineering principle: Feature -> understand data and business rule -> identify security risks -> validate/authenticate/authorize -> implement -> test normal and malicious/invalid cases -> move on.

10. Immediate Development Order

8. Initialize the Next.js + React + TypeScript application and Willis Port branding.
9. Create the MVP data model: User, Customer, ServiceRequest, Shipment/Cargo, PricingRule, ExchangeRate, Invoice and InvoiceItem.
10. Implement customer/request entry.
11. Implement cargo measurements and pricing calculation.
12. Implement invoice creation and persistence.
13. Generate the PDF invoice.
14. Create invoice listing/status screen.
15. Run MVP security and validation checks and prepare the client demonstration.